CSE 465 · Principles of Big Data Management | SRM University–AP | Research target: Q1 journal

Q-Stock

A Big Data framework for financial forecasting with classically simulated quantum walks on market-state graphs — end-to-end project brief: architecture, technology stack, code and usage of every element

R 4.6 | Hadoop · MapReduce · Hive | Spark (sparklyr) | DuckDB · Parquet | CUDA (torch for R) | plumber API | React · three.js | Render · Vercel

Repository	github.com/Nabisahebshaik/Q-Stock (branch `main`, commit `ba25e23`, 17 commits)
Live web UI	https://q-stock-pink.vercel.app
Live R API	https://q-stock-api.onrender.com (interactive docs at `/__docs__/`)
Markets	20 NSE (India) + 20 NASDAQ (US) large caps, daily data 2015-01-01 → 2026-10-21, plus live 1-minute data
Document date	27 October 2026
Scope statement	All quantum walks are **classical simulations** (dense/sparse linear algebra on CPU; CUDA only for a benchmark). No quantum hardware is used and no quantum speed-up is claimed. Synthetic data only validates software.

Contents

1 Executive summary
2 End-to-end system flow
3 Technology stack and what each element is used for
4 Repository layout
5 Research method: graphs, quantum walks, features, evaluation
6 Code reference — R library (`R/`)
7 Code reference — pipeline scripts (`scripts/`)
8 Big Data layer — HDFS, MapReduce, Hive, Spark
9 Storage — files, DuckDB, Parquet, SQLite
10 Serving — R API, Shiny dashboard, React web UI, chart workspace
11 Testing and quality
12 Deployment and operations
13 Results (headline numbers)
14 Key engineering decisions, limitations, open items
15 Development timeline and Claude workspace setup

1 · Executive summary

Question. Do quantum walks — run on graphs of how stocks move together — carry information that helps forecast markets, beyond what their classical counterparts and simple baselines already provide?

Approach. Every trading day, Q-Stock builds a correlation network of 20 stocks per market. It runs classical diffusions and quantum walks (DTQW, CTQW and a *quantum stochastic walk* whose dial ω moves smoothly from fully quantum, ω = 0, to fully classical, ω = 1) on the same graph, turns the walk distributions into per-stock features, and tests them under a leakage-controlled, pre-registered walk-forward protocol with proper statistics.

System. Around this research core sits a complete Big Data and product stack: HDFS + Java MapReduce + Hive on a Cloudera VM, Spark through sparklyr, GPU benchmarks with CUDA, a DuckDB/Parquet live-data store, an R REST API, a Shiny dashboard and a TradingView-style React web app deployed free on Render and Vercel.

No beat
Next-day direction: no feature set beats a naive base-rate predictor in either market (log-loss 0.6930 NSE / 0.6921 NASDAQ).

ω ≈ 0.25 wins
NASDAQ 5-day volatility: partially coherent QSW beats its classical limit (Holm p = 4.8×10^{-6}); not on NSE.

GHAR best
A simple classical graph-HAR model is as good or better; QSW features add nothing on top of it.

5,680 groups
Java MapReduce output matches R exactly (max error 1.8×10^{-15}); Hive consistency query 0 mismatches.

430×
CUDA float32 CTQW at N = 1,600 vs base R (0.145 s vs 62.3 s).

6.6 s → 0.09 s
Cold chart load after moving live candles to DuckDB and serving charts from the store.

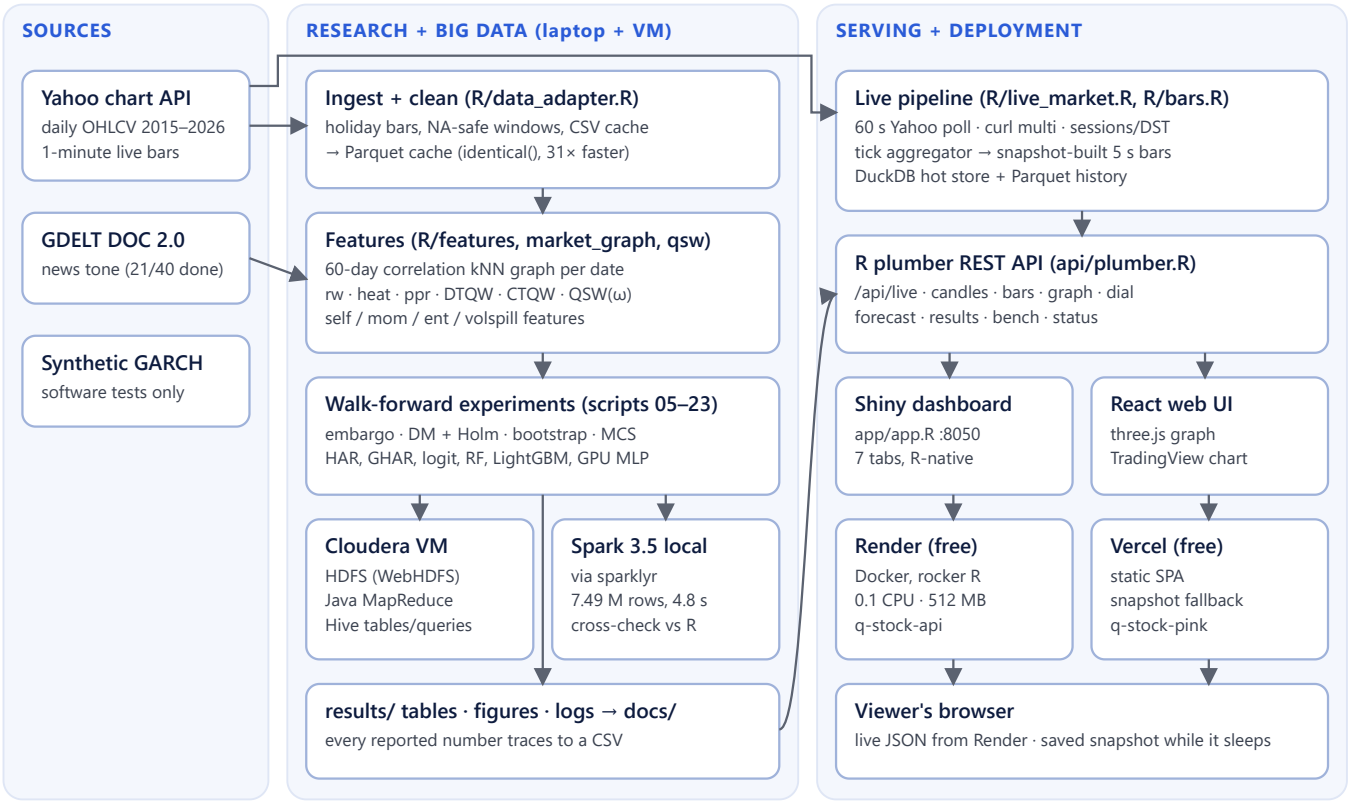
Bottom line. Quantum coherence measurably changes how a walk spreads over the market graph, and on NASDAQ an intermediate level of coherence gives slightly better volatility features than the classical limit of the same walk. But no quantum-walk feature set improves on simple classical baselines. The evaluation had the power to detect weak signals (power analysis), so this is an informative negative result about practical quantum advantage — the project's research contribution is the controlled quantum-to-classical audit itself.

Status (27 Oct 2026)

- All planned experiments complete; results in `docs/results/`, every number traceable to `results/tables/*.csv`.
- Web app and API deployed and verified; checkpoint tag `checkpoint-2026-10-25`.
- Tests: 61 testthat blocks / 357 expectations, 34 API end-to-end checks, Shiny server tests, TypeScript type-check, lint and build — all passing.
- Open: observe 5-second bars during market hours; Render memory check; manuscript writing paused until requested.

2 · End-to-end system flow

The project has three faces that share one R computation library (`R/`): the **research pipeline** (batch), the **Big Data layer** (course component) and the **serving layer** (live product). Every entry point — scripts, tests, Shiny, the API and the Docker image — sources all of `R/*.R` and the shared configuration `QS` .



How a piece of data travels

Stage	What happens	Code
1 · Acquire	Yahoo daily OHLCV for 40 tickers is downloaded once and cached; live 1-minute bars are polled every 60 s for viewing only.	`R/data_adapter.R` , `R/live_market.R`
2 · Clean	Holiday bars dropped, NA-safe rolling windows, extreme moves flagged (not silently removed); quality report written.	`clean_prices()` , `results/data_quality_report.txt`
3 · Graph	For each market and date: 60-day trailing correlations → symmetrised k = 4 nearest-neighbour graph → Laplacian.	`R/market_graph.R`
4 · Walk	Six processes run on the same graph from every start node, giving a distribution matrix P.	`R/dtqw.R` , `R/ctqw.R` , `R/qsw.R`
5 · Features	Identical node features from every P (return probability, momentum spillover, entropy, volatility spillover) joined with classical features and targets.	`R/qsw_features.R` , `R/dataset.R`
6 · Evaluate	Monthly walk-forward with an embargo; models fitted on the past only; DM tests with Holm correction, bootstrap CIs, MCS, power analysis.	`R/walk_forward.R` , `R/evaluation.R`

7 · Scale out	Same data stored in HDFS, aggregated by MapReduce, queried in Hive and recomputed in Spark — all cross-checked against R.	`hadoop/`, `hive/`, `scripts/03–04`
8 · Report	Tables (CSV + Markdown) and 300-dpi figures feed the docs and manuscript.	`scripts/08`, `12`; `docs/results/`
9 · Serve	The API computes live graphs, the quantum dial, forecasts and chart bars; React and Shiny render them; Render and Vercel host them.	`api/`, `app/`, `web/`, `deploy/`

3 · Technology stack and what each element is used for

Versions are the ones installed on the development laptop (R packages) or pinned in `web/package.json` / the Dockerfiles.

3.1 Languages and runtimes

Element	Version	Used for
R	4.6.1 (laptop), 4.4.2 (Docker)	All analysis: data, graphs, quantum-walk simulation, models, statistics, API, dashboard. A course rule: no Python/PySpark.
Java	JDK 1.7 (VM), Temurin 17 (host)	The MapReduce job (Hadoop's native language); Java 17 runs local Spark.
HiveQL	Hive 1.1 (CDH 5.13)	External tables, window-function queries and consistency checks.
TypeScript	5.6	The React web UI; types mirror the API contract.
Bash / PowerShell	Git Bash, PS 5.1	Overnight job chains, VM start-up, thermal guard, system monitor, skill packaging.
SQL (DuckDB / SQLite)	DuckDB 1.5.6	Candle upserts, intraday roll-ups, Parquet export and pruning.

3.2 R packages

Package	Version	Used for	Where
base R, stats	4.6.1	Linear algebra (eigen, matrix products), OLS/logit, RK4 integration of the Lindblad equation	`R/` throughout
Matrix	1.7.5	Sparse DTQW operators for large graphs (benchmarked to N = 1,600)	`R/dtqw.R`
ranger	0.18.0	Random-forest direction classifier (v1)	`scripts/05`, `06`
glmnet	5.1	Elastic-net volatility model with nested tuning	`scripts/20`
lightgbm	4.7.0	Nonlinear gradient-boosted forecaster, nested validation	`scripts/21`
torch (CUDA)	0.17.0	GPU benchmark of CTQW/DTQW; QLIKE-trained MLP on the RTX 4060	`scripts/13`, `18`
sparklyr + dplyr	1.9.6	Driving Spark 3.5: window features, point-in-time range join, scaling runs	`scripts/04`
duckdb + DBI	1.5.6 / 1.3.0	Live candle store, intraday roll-ups, Parquet read/write, price cache	`R/store_duckdb.R`, `R/bars.R`
RSQLite	3.53.3	Original candle store, kept as rollback backend	`R/live_market.R`
curl, jsonlite	8.1.0 / 2.0.0	Concurrent Yahoo/GDELT downloads (curl multi), JSON parsing and API output	`R/data_adapter.R`, `live_market.R`
plumber	1.3.3	REST API, static file serving, hooks (timing, cache headers), CORS filter	`api/plumber.R`
shiny, bslib, DT, plotly, visNetwork	1.14 / 0.12 / … / 4.12 / 2.1	R-native dashboard: tabs, tables, candlesticks, network graphs	`app/app.R`
ggplot2	4.0.3	Research figures (300 dpi)	`scripts/08`, `12`

testthat, callr, httr	3.3.2	Unit tests; background API process and HTTP checks in end-to-end tests	`tests/`
bench, chromote	1.1.4 / 0.5.1	Timing benchmarks; headless-Edge screenshots of the dashboard and web UI	`scripts/07` , `22` , `24` , `31`

3.3 Big Data platform

Element	Version	Used for
Cloudera QuickStart VM	CDH 5.13, CentOS 6.7	The lab environment: 8 GB RAM, 4 vCPU in VirtualBox 7.2, reached over NAT-forwarded SSH (port 2222).
HDFS (WebHDFS REST)	Hadoop 2.6	Stores curated prices, code, experiment results and predictions under `/user/cloudera/qstock` .
YARN + MapReduce	2.6	Runs the Java job computing per ticker-month statistics with a combiner and 2 reducers.
Hive + HiveServer2 + Metastore (MySQL)	1.1	External tables over HDFS CSVs; analytic queries via beeline.
ZooKeeper, Cloudera Manager REST v18	CDH 5.13	Service coordination; starting services programmatically after each boot.
Apache Spark (local)	3.5.9, `local[8]`	Distributed-style feature engineering through sparklyr (the VM's Spark 1.6 is incompatible with sparklyr).
winutils.exe / hadoop.dll	Hadoop 3.3.6 build	Hadoop native IO for Spark on Windows (third-party binaries, hashes recorded).

3.4 Web front end (`web/package.json`)

Library	Version	Used for
React + React DOM	18.3.1	Single-page app with four pages (Live market, Quantum dial, Research, Pipeline)
Vite	5.4	Dev server and builds in three modes: local live, static snapshot, remote live (Vercel)
Tailwind CSS + PostCSS	3.4.14	Styling with CSS variables for light/dark themes
TanStack Query	5.59	API calls, polling (2 s / 60 s / 30 min), caching, keep-previous-data, fallback to snapshot
three.js, @react-three/fiber, drei, postprocessing	0.169 / 8.17 / 9.114 / 2.16	3D quantum market graph with glow, particles along edges
d3-force-3d	3.0.5	Force-directed 3D layout of the 20-stock graph
lightweight-charts	4.2.0	TradingView-style candles, volume and RSI bands, overlays; drawing tools via series primitives
recharts	2.13	Research and dial charts (entry animation disabled — it rendered blank)
framer-motion, lucide-react	11.11 / 0.453	Transitions; toolbar and menu icons
ESLint, typescript-eslint	9.x / 8.x	Lint (react-hooks rules)

3.5 Hosting and tooling

Element	Used for

Docker (`rocker/r-ver:4.4.2`)	API container image; duckdb pinned to 1.5.6 from a dated Posit Package Manager binary snapshot.
Render (free web service)	Hosts the R API from `render.yaml` (Singapore, 0.1 CPU, 512 MB, sleeps after 15 min, auto-deploys from `main`).
Vercel	Hosts the built web UI from `web/vercel.json` ; cache headers; bundled snapshot data for fallback.
Hugging Face Spaces (kept)	Alternative Docker bundle (`deploy/hf-space`), unused since Spaces became paid.
GitHub	Source of truth; Render and Vercel deploy on push; deploy statuses reported back.
NVIDIA RTX 4060 Laptop (8 GB)	CUDA benchmark and MLP training only; experiments on 20-node graphs run on CPU.
Claude Code / Cowork	Development assistant: `CLAUDE.md` rules, 10 project skills, parallel agents for independent work streams.

4 · Repository layout

Folder	Size	Contents and role
`R/`	19 files · 2,016 lines	The computation library shared by every entry point: data, graphs, walks, features, evaluation, live market, storage, bars and indicators.
`scripts/`	40 files · 2,285 lines	Numbered pipeline steps 01–31, overnight orchestration shells, thermal guard and monitoring.
`tests/`	14 files · 993 lines	testthat unit tests, API end-to-end test, Shiny server test.
`api/`	2 files · 423 lines + JSON	plumber REST API, launcher, exported GHAR coefficients, precomputed dial payloads.
`app/`	1 file · 340 lines	Shiny dashboard.
`web/`	29 source files · 3,113 lines	React + TypeScript UI (pages, 3D graph, chart workspace, drawing tools), build scripts, snapshot data.
`hadoop/`, `hive/`	3 + 4 files · 310 lines	Java MapReduce job, VM start-up / HDFS fix scripts; HiveQL DDL and queries.
`deploy/`, `render.yaml`	—	Render and Hugging Face Dockerfiles; Render Blueprint.
`results/`	tables, figures, logs	Every reported number (CSV + Markdown), 9 figures, run logs, screenshots, session info.
`data/`	git-ignored	`raw/` (Yahoo, GDELT cache), `curated/` (features RDS, prices Parquet), `live/` (DuckDB, Parquet, SQLite), `synthetic/`.
`docs/`	architecture, design, research, results, operations, course	Architecture and design documents, pre-registration, novelty audit, results, runbook, issues log, hand-off, CO mapping.
`paper/`	1 file	Manuscript draft (framed as a pre-registered quantum-to-classical audit).
`.claude/`, `CLAUDE.md`	10 skills	Instructions and reusable workflows for Claude sessions.

5 · Research method

5.1 Market-state graph (per market, per trading day t)

1. Daily log returns; Pearson correlations over the trailing 60 days ending at t (all known at the close of t). A stock joins the graph only if ≥ 80% of the window is observed.
2. Each stock links to its 4 most-correlated peers; links are symmetrised. Edge weight = max(ρ, 0.01).
3. Graph Laplacian L = D − W. A **random-graph control** replaces ρ with random values before the same kNN step, separating "network structure" from "pooling across stocks".

5.2 Walk processes (all on the same graph, started from every node)

Process	Definition	Code
Random walk (rw)	Transition $D^{-1}W$, averaged over steps 1–10	`walk_distributions()`
Heat kernel	$\exp(-\tau L)$, averaged over a 10-point τ grid in (0, 2]	`walk_distributions()`
Personalised PageRank	$(1-\alpha)(I - \alpha T)^{-1}$, α = 0.85	`walk_distributions()`
DTQW	Grover-coin discrete-time quantum walk on the arc space of the graph (dense and sparse)	`R/dtqw.R`
CTQW	$\lvert\exp(-i\tau L)\rvert^2$, the closed quantum walk	`R/ctqw.R`
QSW(ω)	Lindblad master equation interpolating quantum (ω = 0) ↔ classical (ω = 1)	`R/qsw.R`

$$d\rho/dt = -(1-\omega)\cdot i[H, \rho] + \omega\cdot\Sigma_{i\neq j}\ \gamma_{ij}\ (\ L_{ij}\ \rho\ L_{ij}^{\dagger} - \tfrac{1}{2}\{L_{ij}^{\dagger}\ L_{ij}, \rho\}\ ),\quad H = L,\quad L_{ij} = |i\rangle\langle j|,\quad \gamma_{ij} = W_{ij}$$

The QSW is integrated with RK4 (step 0.02) for all start states at once as an N×N×N complex array; the dissipator is computed in closed form (diag(W·p) and diag(d)), avoiding N^2 jump operators. Unit tests verify that ω = 0 reproduces the CTQW and ω = 1 the heat kernel to 10^{-6}. The pre-registered grid is ω ∈ {0, 0.1, 0.25, 0.5, 0.75, 0.9, 1}.

5.3 Features and targets

Feature	Meaning
`self`	P[i, i] — how much of the walk returns to the stock (localisation)
`mom`	Walk-weighted mean of neighbours' z-scored 5-day returns (momentum spillover)
`ent`	Shannon entropy of P[i, ·] — how widely influence spreads
`volspill`	Walk-weighted mean of neighbours' log 5-day realised variance (volatility spillover)
Classical	ret_1, ret_5, ret_20, vol_10, vol_20, range, log_vol_z; HAR terms (log RV over 1, 5, 22 days); GHAR adds neighbour averages of the HAR terms

Targets: T1 (primary) log of next-5-day realised variance; T2 next-day direction; T3 cross-sectional rank IC.

5.4 Evaluation protocol

- **Walk-forward:** 24-month rolling training window, one test month per fold (2017-03 → 2026-10). Training rows whose label is revealed inside the test month are dropped (embargo). Scaling and tuning use training data only; nested tuning uses the last 6 months of the training window.
- **Leakage tests:** future rows are perturbed and features asserted unchanged.
- **Statistics:** Diebold–Mariano with Newey–West variance; Holm within pre-registered families, Benjamini–Hochberg for ablations; stationary bootstrap CIs (block 20, 1,000 reps); Model Confidence Set (B = 2,000); exact linear SHAP; power

analysis with injected signals.

- **Pre-registration:** hypotheses H1–H4, ω grid, targets and corrections fixed in `docs/research/preregistration_v2.md` before any v2 run; everything else labelled exploratory.

6 · Code reference — R library (`R/`)

File	Responsibility	Key functions	Used by
`config.R`	Dates, the 40-stock universe, graph and walk-forward settings, column groups	`QS`, `CLASSICAL_COLS`, `GRAPH_METHODS`, `graph_cols()`	everything
`data_adapter.R`	Yahoo download and parsing; cleaning; CSV cache; Parquet price cache (identical to CSV)	`fetch_yahoo_daily`, `clean_prices`, `load_market_data`, `write/read_prices_parquet`	scripts, API, Shiny
`synthetic_data.R`	GARCH price simulator for software validation	`simulate_garch_prices`	tests, stage 1
`features.R`	Classical per-stock features with past-only windows	`trailing_mean`, `trailing_sd`, `build_features`	scripts 02
`market_graph.R`	kNN correlation graph; classical and quantum walk distributions; node features	`knn_graph`, `walk_distributions`, `node_features_from_P`, `returns_wide`	scripts, API, Shiny
`dtqw.R`	Grover DTQW on graph arcs (dense + sparse) and on a cycle	`dtqw_graph_operator(_sparse)`, `dtqw_evolve`, `is_unitary`	features, benchmarks
`ctqw.R`	Continuous-time quantum walk with H = L	`ctqw_propagator`, `ctqw_probs`, `is_hermitian`	features, benchmarks
`qsw.R`	Quantum stochastic walk (Lindblad), batched RK4, exact limits	`qsw_rhs`, `qsw_distributions`, `qsw_trace`, `qsw_purity`	v2 features, API dial
`qsw_features.R`	v2 QSW features per ω, GHAR regressors, RV targets	`qsw_features_market`, `ghar_features_market`, `build_vol_targets`	scripts 10–23
`sentiment_gdelt.R`	GDELT tone timelines, per-ticker cache, point-in-time decay features	`gdelt_timeline`, `sentiment_features`	sentiment chain (RQ5)
`dataset.R`	Modelling table: features, regimes, fusion interactions	`build_dataset`, `add_regime`	scripts 02, 05
`walk_forward.R`	Monthly folds, embargoed split, train-only scaling, fit/predict loop	`walk_forward_folds`, `wf_split`, `walk_forward(_reg)`	all experiments
`baseline_logit.R`	Stage-1 chronological logistic baseline	`chrono_split`, `fit_logit_baseline`	stage 1
`evaluation.R`	Metrics, backtest, DM test, bootstrap, Holm/BH, QLIKE, rank IC	`dm_test`, `stationary_boot_ci`, `holm_adjust`, `qlike`	all experiments
`hadoop_bridge.R`	WebHDFS REST client and VM SSH helper	`hdfs_put`, `hdfs_ls`, `hdfs_get_text`, `vm_ssh`	script 03
`live_market.R`	Exchange sessions, Yahoo 1-min polling, candle-store dispatch, tick aggregator, live graph, live GHAR forecast, snapshot fetcher	`market_status`, `fetch_intraday_many`, `candle_upsert`, `tick_aggregator`, `live_graph`, `ghar_live_forecast`	API, Shiny
`store_duckdb.R`	DuckDB candle store: bulk idempotent upsert, range reads, Parquet export, version-portable driver	`duck_drv`, `duck_store_open`, `duck_upsert`, `duck_read`, `duck_export_parquet`	API, Shiny, scripts 29–31
`bars.R`	Chart bars: intraday roll-ups (DuckDB SQL and base R), daily/weekly/monthly roll-ups, range rules, snapshot-built 5 s bars	`rollup_intraday_duck`, `rollup_daily`, `range_start`, `snapshot_tick`, `snapshot_bars`	`/api/bars`

`indicators.R`	Technical indicators computed in R with warm-up history; request validation	`sma`, `ema`, `bollinger`, `rsi`, `vwap_session`, `parse_ind_spec`, `compute_indicators`	`/api/bars`

7 · Code reference — pipeline scripts (`scripts/`)

Script	Purpose	Main output
`01_stage1_prototype.R`	DTQW/CTQW demos and a classical logistic baseline	`results/stage1_*`
`02_build_dataset.R`	Real data → cleaned prices → classical + graph-walk features	`data/curated/dataset_graph.rds`
`03_hadoop_hive.R`	HDFS upload, MapReduce run, Hive tables and queries (needs the VM)	`hive_*.tsv`, stage-3 log
`04_spark_sparklyr.R`	Spark window features, point-in-time join, scaling, R cross-check	`spark_scaling.csv`
`05_experiments.R`, `05b`	v1 direction experiments; naive base-rate reference	`experiment_results.csv`
`06_ablations.R`	Ablations and hyper-parameters	`ablation_results.csv`
`07_scalability.R`	CPU scalability of the walk simulations	`qw_scalability.csv`
`08_figures.R`, `12_figures_v2.R`	300-dpi figures and Markdown tables	`results/figures/`
`09_literature_survey.R`	Structures the 45-paper student survey for the novelty audit	`docs/research/*.csv`
`10_qsw_features.R`	QSW features for the pre-registered ω grid + random-graph controls (parallel)	`qsw_features.rds`
`11_v2_experiments.R`	Pre-registered H1–H3 tests	`v2_*.csv`
`13_gpu_benchmark.R`	GPU vs CPU timing of the walks (torch, CUDA)	`gpu_benchmark.csv`, fig 8
`14`, `15`, `16`	Robustness (exploratory), exact SHAP, economic value	`v2_robustness_*`, `explainability_*`
`17_ghar_mcs.R`	Graph-HAR baseline and Model Confidence Set	`v2_ghar_*.csv`, `v2_mcs.csv`
`18`, `23`	QLIKE-trained GPU MLP; comparison with linear models on identical rows	`v2_mlp_qlike_gpu*.csv`
`19_power_analysis.R`	Positive control: can the pipeline detect a real signal?	`power_analysis*.csv`
`20`, `21`	Nested-tuned models (adaptive ω, elastic net, combination); LightGBM	`v3_*.csv`
`22`, `24`	Headless-Edge screenshots of the Shiny dashboard and the web UI	`results/screenshots/`
`25_export_snapshot.R`	Calls every API endpoint and saves JSON for static hosting (211 files)	`web/public/data/`
`26_export_ghar_coef.R`	Exports live graph-HAR coefficients for the container	`api/model/*.json`
`27_build_hf_space.sh`	Assembles the Hugging Face Space bundle	`build/hf-space/`
`28_package_skills.ps1`	Zips each Claude skill for Cowork upload	`build/skills/*.zip`
`29_migrate_sqlite_to_duckdb.R`	Verified SQLite → DuckDB migration, Parquet export, compaction	`data/live/candles.duckdb`
`30_prices_to_parquet.R`	Builds and verifies the Parquet price cache	`data/curated/prices.parquet`
`31_storage_benchmark.R`	SQLite vs DuckDB vs Parquet query timings	`storage_benchmark.csv`

`overnight_v2.sh`, `after_*.sh`, `resume_gpu_chain.sh`, `overnight_sentiment_chain.sh`	Unattended job chains with `=== step OK/FAIL` markers and explicit exit-status checks	`results/overnight_*.log`
`thermal_guard.ps1`, `monitor_system.ps1`	Pause R/Java and the VM at 87 °C, resume ≤ 65 °C; 5-minute health log	`results/thermal_guard.log`
`fetch_sentiment_overnight.R`	Patient GDELT download with per-ticker cache and retries	`data/raw/gdelt_cache/`

8 · Big Data layer

HDFS (via WebHDFS from R)

`R/hadoop_bridge.R` talks to the NameNode REST API (`http://127.0.0.1:50070/webhdfs/v1`) to create folders, upload, list, read and delete files under `/user/cloudera/qstock`: `curated/prices`, `raw/sentiment`, `code`, `results/…`. Replication 1, 128 MB blocks.

Fault fixed: Cloudera Manager wrote 127.0.0.1 into the DataNode include list while the DataNode registers as 10.0.2.15 → 0 live DataNodes, NameNode in safe mode. `hadoop/vm_start_services.sh` re-applies the fix on every boot.

MapReduce (Java)

`hadoop/QStockMonthlyStats.java`: mapper → **combiner** → 2 reducers with custom counters, computing per ticker-month statistics on YARN. Compiled and run in the VM over SSH. Validation: **5,680 groups**, count difference 0, max error **1.8×10^{-15}** vs an independent R computation.

Hive (HiveServer2 :10000, MySQL metastore)

`hive/01_market_tables.hql` and `02_results_tables.hql` create external tables `prices`, `mr_monthly_stats`, `sentiment_daily`, `experiment_results`, `predictions`. `03/04_*.hql` run GROUP BY, `ROW_NUMBER` / `RANK` window functions, joins, and a MapReduce-vs-raw consistency query (**0 mismatches**). Run through beeline over SSH.

Spark 3.5 through sparklyr

sparklyr 1.9.6 cannot drive the VM's Spark 1.6 / Java 7, so Spark 3.5.9 runs locally (`local[8]`, Java 17, 8 GB driver memory). `scripts/04` computes rolling features with dplyr window functions and a point-in-time sentiment range join in Spark SQL, agrees with R to 10^{-15} (features) and 6.5×10^{-7} (sentiment decay), and scales to **7.49 M rows in 4.8 s**.

VM operations

```
VBoxManage startvm "cloudera-quickstart-vm-5.13.0-0-virtualbox" --type headless
ssh -i ~/.ssh/qstock_vm_rsa -p 2222 cloudera@127.0.0.1 'bash -s' < hadoop/vm_start_services.sh
```

The start-up script fixes the include list, starts ZooKeeper and Hive through the Cloudera Manager REST API, and restarts HiveServer2 if it came up before the metastore. The Cloudera Management Service is kept stopped (with it running the guest hung). Thread budget on the 20-thread host: ranger 4, Spark 8, VM 4 vCPU.

Course-outcome coverage (CSE 465)

Course topic	Evidence in Q-Stock
Big Data concepts	Volume (40 tickers × 11.8 years, scaled to 7.5 M rows), variety (prices + news tone), velocity (60 s live feed, 5 s bars)
HDFS	WebHDFS client, block/replication inspection, DataNode fault diagnosis
Hadoop / MapReduce	Java job with combiner, counters, 2 reducers, validated against R
Ecosystem	YARN, Hive Metastore, HiveServer2/beeline, ZooKeeper, Cloudera Manager API
HiveQL	External tables, OpenCSVSerde, window functions, joins, consistency check
Spark	sparklyr pipeline, Spark SQL range join, scaling study
Analytics / ML in R	Quantum-walk simulation, OLS/logit/RF/elastic net/LightGBM/MLP, walk-forward, DM/Holm/bootstrap

9 · Storage — files, DuckDB, Parquet, SQLite

Data	Format / store	Produced by
Daily prices (40 tickers)	`data/raw/prices_yahoo.csv` + `data/curated/prices.parquet` (ZSTD, 2.4 MB vs 12.7 MB CSV, loads 0.03 s vs 0.95 s, `identical()` to the CSV frame)	`load_market_data()`, script 30
Research features	RDS: `dataset_graph.rds`, `qsw_features.rds`, `ghar_features.rds`, `vol_targets.rds`	scripts 02, 10
Results	CSV + Markdown in `results/tables/`; PNG figures; prediction dumps (git-ignored)	scripts 05–23
Live 1-minute candles (hot week)	DuckDB `data/live/candles.duckdb` (API) and `candles_app.duckdb` (Shiny) — one writer process per file	`/api/live`, Shiny poll
Live candle history	Hive-partitioned Parquet `data/live/parquet/interval_sec=60/day=YYYY-MM-DD/`, read with partition pruning	`duck_export_parquet()`, script 29
5-second bars	Same DuckDB table with `interval_sec = 5`, source `yahoo_snapshot`	`snapshot_tick()`
Rollback store	SQLite `data/live/candles.sqlite` (untouched by the migration)	`options(qstock.store_backend = "sqlite")`
Static snapshot	211 JSON files (12.3 MB) in `web/public/data/`	script 25

Candle table (same DDL on DuckDB and SQLite; PostgreSQL-compatible)

```
CREATE TABLE candles (ticker TEXT, interval_sec INTEGER, bucket_start BIGINT,
  open, high, low, close DOUBLE PRECISION, volume DOUBLE PRECISION, source TEXT, updated_at BIGINT,
  PRIMARY KEY (ticker, interval_sec, bucket_start),
  CHECK (high >= low AND high >= open AND high >= close AND low <= open AND low <= close));
-- writes: INSERT … SELECT FROM registered frame ON CONFLICT (…) DO UPDATE  (idempotent, bulk)
```

Measured storage engines (`storage_benchmark.csv`, median of 20)

Query	SQLite	DuckDB	Parquet
One ticker, latest session (391 rows)	**0.62 ms**	1.23 ms	5.6 ms
One ticker, 5 sessions	1.97 ms	**1.36 ms**	6.2 ms
One market, latest session (7,236 rows)	7.28 ms	**3.15 ms**	8.0 ms
Daily roll-up of 1-min candles	83 ms	**6.3–13 ms**	8.7–14 ms
Daily prices load (117,660 rows)	CSV 367–950 ms		**25–30 ms**

> Honest reading: for a single-session chart both engines are sub-millisecond-scale. The user-visible win (cold chart 6.6 s → 0.04–0.09 s) came from serving charts from the store instead of waiting for a Yahoo fetch; DuckDB pulls ahead as queries grow (multi-session, whole market, aggregates).

10 · Serving — R API, Shiny dashboard, React web UI

10.1 REST API (`api/plumber.R`, run with `Rscript api/run_api.R [port]`)

All endpoints are read-only `GET`, return JSON, carry a `Server-Timing` header and allow any origin (CORS). Bad input returns 400. The API also serves the built web UI from `web/dist`.

Endpoint	Returns	Caching / source
`/api/status`	Both markets: open flag, local time, time zone, tickers	—
`/api/live/<market>`	Quotes, change, intraday RV, sparklines, store statistics, stale flag	One shared Yahoo batch per minute
`/api/candles/<m>/<ticker>?days=`	1-min OHLCV + VWAP (reset per session), exchange-local time	DuckDB store + forming bar
`/api/bars/<m>/<ticker>?interval&range&ind`	Bars for 5s, 1m, 5m, 15m, 30m, 1h, 1D, 1W, 1M and ranges 1D…ALL, plus indicator lines (VWAP, SMA, EMA, BB, RSI)	Store, DuckDB roll-ups, daily history, 5 s snapshot aggregator
`/api/graph/<m>?omega&look`	Live correlation graph of 1-min returns with QSW features and the P matrix	Per live batch
`/api/dial?market&date&omega`	Historical daily graph at ω plus the quantum (ω=0) and classical (ω=1) limits	Precomputed for latest date
`/api/forecast/<m>`	Graph-HAR 5-day volatility forecast per stock	30 min
`/api/results`	Research tables read from `results/tables` (nothing recomputed)	—
`/api/bench`	Speed check of the host (20-node QSW)	—

10.2 Shiny dashboard (`app/app.R`, port 8050)

bslib navbar with seven tabs — Overview, Live market (plotly candlesticks + visNetwork live graph + risk table), Quantum dial (live Lindblad integration on a chosen day), Volatility forecasts, Results (pre-registered tests, GHAR + MCS, robustness, power, direction, figures), Big Data pipeline, System health. `?tab=` deep links; screenshots with `scripts/22`.

10.3 React web UI (`web/src/`)

Element	Usage
`pages/LivePage.tsx`	Market switch, KPI cards, 3D quantum market graph with ω slider, selected-stock panel, chart workspace, live risk table
`pages/DialPage.tsx`	Historical quantum dial: sweep ω between quantum and classical distributions for any date
`pages/ResearchPage.tsx`	Every headline result from `/api/results` as charts and tables
`pages/PipelinePage.tsx`	The Big Data pipeline and its validation numbers
`components/QuantumGraph3D.tsx`	react-three-fiber scene: nodes sized by volatility, glow by spillover, particles flowing in proportion to walk transition probabilities
`lib/api.ts`	Typed client and TanStack Query hooks; live/snapshot/fallback logic (`load(live, snap)`)
`App.tsx`	Routing, theme, recovery from stale chunks after a redeploy

10.4 TradingView-style chart workspace (`web/src/components/chart/`)

Component	Elements and usage

`TradingWorkspace.tsx`	Holds state (ticker, interval, range, chart type, indicators, scale); range sets a default interval; persists preferences; screenshot, fullscreen, go-to-date, zoom
`ChartToolbar.tsx`	Symbol search · interval menu (5s "snapshot-built", 1m–1h, 1D/1W/1M) · chart type (candles, hollow, bars, line, area, baseline) · Indicators · undo/redo · settings (scale, grid, crosshair) · fullscreen · screenshot
`TradingChart.tsx`	lightweight-charts: main series, volume band, RSI band with 30/70 lines, SMA/EMA/VWAP/BB overlays, log/percent scale
`ChartLegend.tsx`	"TICKER · interval · market", O/H/L/C and change for the hovered bar; indicator rows with eye, gear, trash and more-menu icons
`IndicatorsDialog.tsx`	Add VWAP, SMA, EMA, Bollinger, RSI, Volume; edit length, k and colour; show/hide; remove
`BottomBar.tsx`	Ranges 1D · 5D · 1M · 3M · 6M · YTD · 1Y · 5Y · All, calendar, live exchange clock (e.g. UTC+5:30), % / log / auto, source badges
`drawing/` (manager, toolbar)	Left toolbar: cursor, crosshair, trend line, ray, horizontal line/ray, vertical line, rectangle, Fibonacci, measure, text; magnet, lock, hide, remove all, zoom; undo/redo; drawings saved per stock in the browser

> Indicator values are computed in R (project rule: the UI only renders), and were cross-checked by hand on real data: daily SMA 20 = 1,192.59, Bollinger upper = 1,231.68, 1-min VWAP = 1,180.66, RSI 14 = 77.23 (RELIANCE.NS).

11 · Testing and quality

Suite	Run with	What it proves
Quantum core	`test_dtqw/ctqw/qsw.R`	Unitarity, Hermiticity, probability conservation, QSW limits vs closed forms (10^{-6}), sparse = dense
Pipeline	`test_stage2/baseline/v2.R`	Leakage (perturb future → features unchanged), embargo, holiday cleaning, NA-safe windows, metrics
Live and storage	`test_live.R`, `test_store_duckdb.R`, `test_prices_parquet.R`	Sessions, idempotent upsert, rejected bars, tick aggregator ordering and late ticks, DuckDB = SQLite, Parquet round-trip identical, no duplicate export
Bars and indicators	`test_bars.R`, `test_indicators.R`	Known-value SMA/EMA/BB/RSI/VWAP, spec validation, roll-up DuckDB = base R across the US DST change, 5 s aggregator with mocked snapshots
All unit tests	`Rscript tests/run_tests.R`	**61 blocks / 357 expectations, 0 failed**
API end-to-end	`Rscript tests/test_api.R`	**34 checks** with a mocked feed: every endpoint, errors, caching, cold store, static serving
Dashboard	`Rscript tests/test_app.R`	Shiny server logic with mocked fetchers
Web	`npx tsc -b`, `npm run lint`, `npm run build`	Types match the API contract; 0 lint errors; production build
Big Data	in-script cross-checks	MapReduce vs R (1.8×10^{-15}), Hive consistency (0 mismatches), Spark vs R (10^{-15})
Research integrity	doc-vs-CSV audits	Every number in docs, README and manuscript traced to its CSV; corrections logged in the hand-off

12 · Deployment and operations

12.1 Deployment topology

Part	How it is built and run
R API on Render	`render.yaml` → `deploy/render/Dockerfile` (rocker/r-ver 4.4.2; plumber, RSQLite, DBI, curl, jsonlite, Matrix; duckdb 1.5.6 pinned from a dated binary snapshot with a version check). Copies only `R/`, `api/`, `results/tables`; prices download at build. `OPENBLAS_NUM_THREADS=1`; no HTTP health check. Auto-deploys on push to `main`.
Web UI on Vercel	`web/vercel.json` → `npm run build:vercel` (live build against `VITE_API_URL` from `web/.env.production`), output `dist-vercel`; assets cached 1 year, data 5 min.
Fallback	While the free API sleeps or fails, the UI loads the bundled snapshot (`/data/*.json`), shows a notice, and switches back to live data automatically.

12.2 Measured on the deployed services (25 Oct 2026)

Request	Time
Chart bars from the browser (1m, 1D)	0.64 s
`/api/bars` daily 1Y with SMA, Bollinger, RSI	2.4 s
First 1m/5m request after a restart (one Yahoo fetch into an empty store)	8.6–11 s
`/api/candles` from the store	0.45 s
Earlier free-tier findings: live quotes ≈ 11 s, graph ≈ 6 s, forecast ≈ 5 s, dial 0.1–4 s; ≈ 230 MB of 512 MB	—

> Incident log: the first deploy of the chart workspace crashed at start-up because the image installed duckdb 1.2.0, which rejects the `shared_home` argument used locally with 1.5.6. Fixed by a version-portable driver (`duck_drv()`) and pinning duckdb 1.5.6 in the Dockerfiles; the public site stayed usable via the snapshot fallback meanwhile.

12.3 Everyday commands

```
# R is not on PATH:  & "C:\Program Files\R\R-4.6.1\bin\x64\Rscript.exe"
Rscript tests/run_tests.R                      # unit tests
Rscript tests/test_api.R                       # API end-to-end (offline)
cd web && VITE_API_URL= npm run build          # UI for the local API (plain build targets Render)
Rscript api/run_api.R                          # API + UI on http://127.0.0.1:8000
Rscript -e "shiny::runApp('app', port = 8050)"  # Shiny dashboard
Rscript scripts/25_export_snapshot.R           # refresh the static snapshot
bash scripts/overnight_v2.sh                   # unattended research re-run
```

Long runs write `=== step OK/FAIL` markers and check every R exit status; the thermal guard (`scripts/thermal_guard.ps1 -Hours 14`) pauses work at 87 °C.

13 · Results (headline numbers)

Source: `docs/results/results.md` and `results_v2.md`; every value comes from `results/tables/*.csv`.

Finding	NSE	NASDAQ
Direction: naive base-rate log-loss	**0.6930**	**0.6921**
Direction: best model log-loss	0.6937	0.6930
Direction: AUC range (16 runs)	0.5025–0.5070	0.5012–0.5123
H2 (pre-registered): ω < 1 beats classical ω = 1 for 5-day volatility	Not supported (classical best)	**Supported**: ω = 0.25, Δ = −0.0037, 95% CI [−0.0052, −0.0019], Holm p = 4.8×10^{-6}; ω = 0.5 Holm p = 4.7×10^{-8}; holds under QLIKE, both halves of the sample, 17/20 stocks
H3: random-graph control reproduces the gain?	Yes (pooling effect)	No (p = 0.29)
Best achievable forecast (nested tuning)	Combination, OOS R^2 0.178	Linear GHAR, OOS R^2 0.348
QSW features on top of GHAR	No gain	No gain (worse under QLIKE, p = 0.004)
LightGBM vs GHAR	Worse (R^2 0.144)	Worse (R^2 0.329, p = 8.6×10^{-9})
GPU MLP (QLIKE-trained) vs GHAR on QLIKE	Worse (Holm p = 0.001)	Tie (p = 0.78)
Economic value (inverse-volatility portfolio)	Sharpe changes ≤ 0.001 between ω = 0.25 and ω = 1 — no material benefit	
Power analysis	Detects corr ≈ 0.05	Detects corr ≈ 0.035 (0% false positives)
RQ5 sentiment	Not evaluated: GDELT stopped at 21/40 tickers (HTTP 429); partial data not used by design	

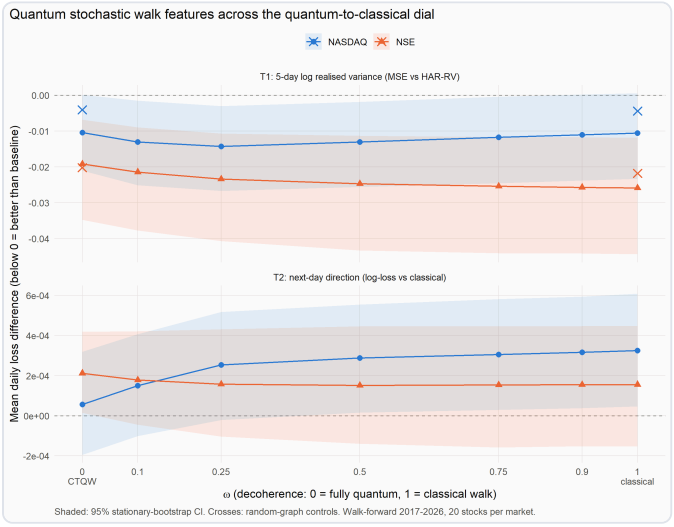


Figure 7 · The quantumness dial: loss difference vs ω (scripts/12).

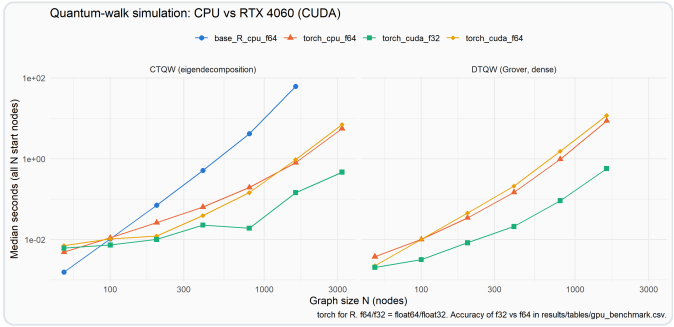


Figure 8 · GPU vs CPU walk simulation time (scripts/13).

GPU benchmark (N nodes)	base R f64	torch CPU f64	CUDA f32
CTQW, N = 1,600	62.3 s	0.804 s	**0.145 s**
CTQW, N = 3,200	—	5.62 s	**0.466 s**
DTQW, N = 1,600	—	8.68 s	**0.572 s**

CUDA float64 is not faster than the CPU on this consumer GPU; below ~200 nodes the GPU has no advantage, so the study's 20-node graphs ran on CPU.

14 · Key engineering decisions, limitations, open items

Decision	Reason
One walk family with a quantumness dial (QSW, ω)	Isolates coherence on an identical graph — v1 compared different feature families and could not.
Pre-register v2 before running it	Prevents tuning for significance; exploratory work is labelled.
Spark 3.5 locally via sparklyr	sparklyr has no backend for the VM's Spark 1.6 / Java 7.
Java MapReduce, R everywhere else	Course rule (R only, no PySpark); Java is Hadoop's native language.
R reaches the VM via WebHDFS + SSH/beeline	The CentOS 6.7 guest has no R; keeps the lab VM stock.
Live market built in R (curl multi + DuckDB)	R-only rule, no paid tick feed, memory headroom next to the 8 GB VM.
Computation only in R; UI only renders	One source of truth for the maths; indicators, roll-ups and walks are tested once.
Static snapshot + live fallback	Vercel cannot run R; the free API sleeps; the site must never be blank.
Render with one BLAS thread, no health check, precomputed dial	Measured: thread contention made a 20-node QSW 39 s (2.0 s single-threaded); the health check killed long requests.
DuckDB + Parquet for live candles; Parquet price cache	Charts served from the store (6.6 s → 0.09 s cold), faster multi-session/aggregate queries, 31× faster price loads.
Snapshot-built 5 s bars, labelled as such	Yahoo has no sub-minute data and no free tick feed; honest labelling instead of fabricated ticks.
Drop RQ5 rather than use partial GDELT data	Pre-stated precondition; partial data would bias the comparison.

Limitations

- Survivorship bias (today's large caps); Yahoo is unofficial and back-adjusted; one pre-registered graph construction; small effect sizes.
- Economic value tested only with an inverse-volatility portfolio; sentiment (RQ5) not evaluated.
- Live tab: exchange holidays not modelled; tick aggregator validated on synthetic/replayed ticks; 5 s bars build only while watched during market hours.
- Chart: lightweight-charts 4 has no separate panes (RSI/volume are bands); intraday history limited to what the store holds.
- Official CSE 465 CO statements still to be inserted; the novelty audit is preliminary (no systematic Scopus/IEEE search yet).

Open items

1. Observe 5-second bars during NSE hours (09:15–15:30 IST).
2. Check Render memory under DuckDB (dashboard → Metrics).
3. Optional: automatic periodic Parquet export inside the API; lightweight-charts 5 panes; per-drawing styles.
4. Manuscript: resolve every [VERIFY] reference and rewrite the abstract with the audit framing — paused until requested.

15 · Development timeline and Claude workspace setup

Commit	Milestone
`86d810b`	Q-Stock core: quantum-walk market graphs for volatility forecasting (R, Hadoop, Hive, Spark, CUDA)
`396cce5`	Live market tab: R live pipeline, hot/cold candle store, tick aggregator, live QSW graph
`8add13b`	React + three.js web UI on an R plumber API; final GPU MLP results; RQ5 closed
`56dab81`	Fix blank Research and Dial charts (recharts animation)
`60591a6` → `9f9f3c7`	Live deployment: Hugging Face bundle, Render Docker service, free-tier fixes (restarts, BLAS threads), Vercel live mode with snapshot fallback, stale-page recovery
`0656737`	Claude session setup, 8 project skills, architecture and design documents, docs reorganised
`4099957`	Live candle store on DuckDB + Parquet; charts read the store; Parquet price cache
`3788ebd`	TradingView-style chart workspace: `/api/bars`, R indicators, drawing tools
`36871df`	Render start-up fix for the image's older duckdb
`b411144`	Checkpoint 2026-10-25 (tag `checkpoint-2026-10-25`)
`ba25e23`	Cowork setup refresh: storage and chart skills, docs brought up to date

Claude Code / Cowork setup

`CLAUDE.md` at the root gives every session the ground rules (R only, classical simulation only, faithful reporting, the 87 °C rule), the environment, commands and conventions. `.claude/skills/` holds ten reusable workflows; `scripts/28_package_skills.ps1` zips them for upload, and `docs/operations/COWORK_SETUP.md` describes the steps.

Skill	Covers
`quantum-walk-simulation`	DTQW/CTQW/QSW in R, features, physics invariants, GPU limits
`leakage-safe-forecast-evaluation`	Point-in-time features, embargoed walk-forward, DM/Holm/bootstrap/MCS, pre-registration
`hadoop-hive-spark-pipeline`	VM start-up and fixes, WebHDFS, MapReduce, Hive, sparklyr
`duckdb-parquet-storage`	DuckDB store, Parquet history and cache, migration, benchmarks, portable drivers
`live-market-data-pipeline`	Sessions, Yahoo polling, candle store, tick aggregator, live analytics
`chart-workspace-bars-indicators`	`/api/bars`, roll-ups, R indicators, React chart workspace and drawings
`r-plumber-react-dashboard`	API patterns, React UI modes, Shiny
`free-tier-r-deployment`	Render + Vercel, free-tier fixes, release checklist
`gpu-and-long-run-safety`	torch/CUDA, overnight chains, thermal guard
`research-results-audit`	Doc-vs-CSV audits, novelty audit, hand-off notes

Further reading in the repository: `docs/architecture/ARCHITECTURE.md`, `docs/design/DESIGN.md`, `DUCKDB_MIGRATION.md`, `CHART_WORKSPACE.md`, `docs/operations/runbook.md`, `docs/results/results.md`, `paper/manuscript_draft.md`.